# 深度学习自学 · 复习要点

> 覆盖 Day 1–11（2026-09-16 ~ 2026-10-18）
> 用途：随时翻看的精简复习资料。配套代码与每日详细日志在 `~/Desktop/AI自学/` 里。

## 一、路线总览

| 阶段 | Day | 主题 | 一句话核心 |
|---|---|---|---|
| 工具 | 1 | 摸底 + 环境 | conda 环境 / GPU(MPS) / PyCharm |
| 工具 | 2 | NumPy | 广播 + axis |
| 工具 | 3 | 线性回归（手写） | 梯度下降 = 深度学习引擎 |
| 工具 | 4–5 | KNN + Git | 向量化距离 + 投票 + 版本管理 |
| PyTorch | 6 | 张量 + 自动求导 | `backward()` 自动算梯度 |
| PyTorch | 7 | nn / optim / DataLoader | 标准 4 步训练循环 |
| PyTorch | 8 | 逻辑回归 | sigmoid + BCE + 决策边界 |
| PyTorch | 9 | MLP | 隐藏层 + ReLU 打破线性 |
| PyTorch | 10 | 过拟合 | train/val gap 检测 |
| PyTorch | 11 | 正则化 + 早停 | 防过拟合两招 |

## 二、Phase 1：工具与传统 ML

### NumPy（Day 2）

- ndarray 三要素：`shape` / `ndim` / `dtype`；切片左闭右开 `a[2:7:2]`
- **广播规则**：两个形状从右往左对齐，逐维比较，满足「相等 / 是 1 / 缺失」即可广播
- **axis**：`axis=0` 跨行运算（对每一列算），`axis=1` 跨列运算（对每一行算）
- 一维数组 shape 是 `(n,)`，不是 `(n,1)`

### 线性回归 + 梯度下降（Day 3）

- 模型 `y = wx + b`；损失 `MSE = mean((y − ŷ)²)`
- 梯度：`dw = 2·mean(err·x)`，`db = 2·mean(err)`
- 更新：`w -= lr·dw` —— **lr 决定速度、不决定终点**；太小慢、太大发散
- 噪声决定精度上限（loss 有下限，学不到 0）

### KNN（Day 4–5）

- 距离矩阵向量化：`(n_test,1,d) − (1,n_train,d)` → `(n_test,n_train,d)` → 平方求和开根
- `argsort` 取 k 近邻 + `bincount` 多数投票

- k 太小 → 过拟合噪声；k 太大 → 边界模糊

### Git（Day 5）

- `init / add / commit`；工作区 → 暂存区 → 版本库
- `core.quotepath false` 才能正确显示中文文件名

---

## 三、Phase 2：PyTorch 深度学习

### 张量 + 自动求导（Day 6）

- Tensor ≈ ndarray + GPU + autograd
- `requires_grad=True` → `backward()` → `.grad`
- 梯度会**累加**，每轮必须 `zero_grad()`
- 叶子变量不能原地改，用 `with torch.no_grad():`

### 标准训练循环（Day 7）

```
for epoch in range(epochs):
    optimizer.zero_grad()              # 1 清梯度
    loss = loss_fn(model(X), y)        # 2 前向 + 损失
    loss.backward()                    # 3 反向传播
    optimizer.step()                   # 4 更新参数
```

- `DataLoader(dataset, batch_size, shuffle)` 分批喂数据

### 逻辑回归（Day 8）

- sigmoid：`σ(z) = 1/(1+e^(-z))`，把 logits 压到 (0,1)
- 损失 BCE：`-[y·log p + (1-y)·log(1-p)]`
- 预测：`p >= 0.5` 判 1 类；决策边界 = `z=0` 的直线

### MLP（Day 9）

- 单层 `Linear` 只能画直线 → XOR 线性不可分
- `Linear → ReLU → Linear` 打破线性
- **每个隐藏神经元 = 一道折痕**：ReLU 在 `w·x+b=0` 处把平面折一下
- 输出 = 折痕加权相加；**决策边界每穿过一道折痕就拐一个弯** → 神经元越多越绕
- 1 个神经元边界仍是直线（折痕 // 边界）；≥2 个才开始折
- hidden 太小可能卡局部最优（「能表示 ≠ 能学到」）

### 过拟合（Day 10）

- `randperm` 打乱后切 train/val（**不打乱，训练集会全是同一类！**）
- **过拟合 = 训练好、验证差**，用 gap = train − val 检测

- gap 本质：大模型**背噪声**，噪声换一批（验证集）就失效
- 数据完全可分 → 训练验证都 100% → gap=0 → 过拟合「隐身」

### 正则化 + 早停（Day 11）

- **weight decay（L2）**：loss 加 `λ·Σw²`，梯度多一项 `2λw`（把权重往 0 拉）
- 本质 = 对大权重按**平方**征税 → 抹平「背噪声」的陡峭拐点
- 为什么平方：L1 常数拉力（稀疏）、L2 线性拉力（温和）、L3 太极端
- **λ（超参数）**：太小 → 过拟合；太大 → **欠拟合**（权重压死，训练验证都差）
- **early stopping**：验证 loss 连续 `patience` 轮不创新低就停 + 回退历史最优
- patience 太小 → 可能误停；太大 → 只费时间（回退最优保护质量）

---

## 四、核心公式速查

| 名称 | 公式 |
| --- | --- |
| sigmoid | σ(z) = 1/(1+e^(−z)) |
| BCE | −[y·log p + (1−y)·log(1−p)] |
| ReLU | max(0, x) |
| MSE | $(1/n)\cdot\Sigma(y-\hat{y})^2$ |
| 线性回归梯度 | dw = 2·mean(err·x) |
| L2 惩罚 | $\lambda\cdot\Sigma w^2$，梯度 2λw |
| 梯度下降 | θ ← θ − lr·∇L |

---

## 五、关键心智模型（记住这些就抓住了主线）

1. **广播**：形状从右对齐，逐维比（相等 / 1 / 缺失）
2. **axis**：0 跨行（对列），1 跨列（对行）
3. **ReLU 折痕**：神经元 = 折痕，边界穿折痕就拐弯
4. **过拟合 = 背噪声**：换一批数据就露馅
5. **weight decay = 按平方征税**：专打大权重
6. **训练 4 步**：zero_grad → forward/loss → backward → step
7. **超参数**（lr、λ、patience、hidden）要靠验证集试出来，不能由训练自动学

---

## 六、高频踩坑清单

- `@`（矩阵乘）vs `*`（逐元素）
- 中文逗号 `0,5` → 被当成元组，报错
- 一维数组 `(n,)` vs 列向量 `(n,1)`

- 忘了 `zero_grad()` → 梯度累加
- 叶子变量原地改 → 用 `no_grad()`
- 切 train/val 忘了打乱 → 训练集全是一类
- 对比实验忘了 `torch.manual_seed(0)` → 结果被随机性污染

---

## 七、进度 & 下一步

- ✅ 已完成：Day 1–11（Phase 1 工具 + Phase 2 PyTorch 基础）
- ⏭️ 下一步：Day 12 —— test 集划分 + 交叉验证（怎么更可靠地评估/选择模型）